
TP - Optimisation numérique et sciences des données

Table of Contents

1. Construction du jeu de données	1
1a. Simulation du signal d'entrée triangulaire	1
1b. Simulation du filtre gaussien	2
1c. Simulation du problème direct par convolution	3
1d. Simulation matricielle du problème direct	5
2a. Déconvolution naïve	5
2b. Déconvolution par moindres carrés	5
2c. Déconvolution par régularisation linéaire quadratique	6

Résolution d'un problème inverse : déconvolution d'un signal triangulaire Auteur : Oussama Elamine Date : Janvier 2025

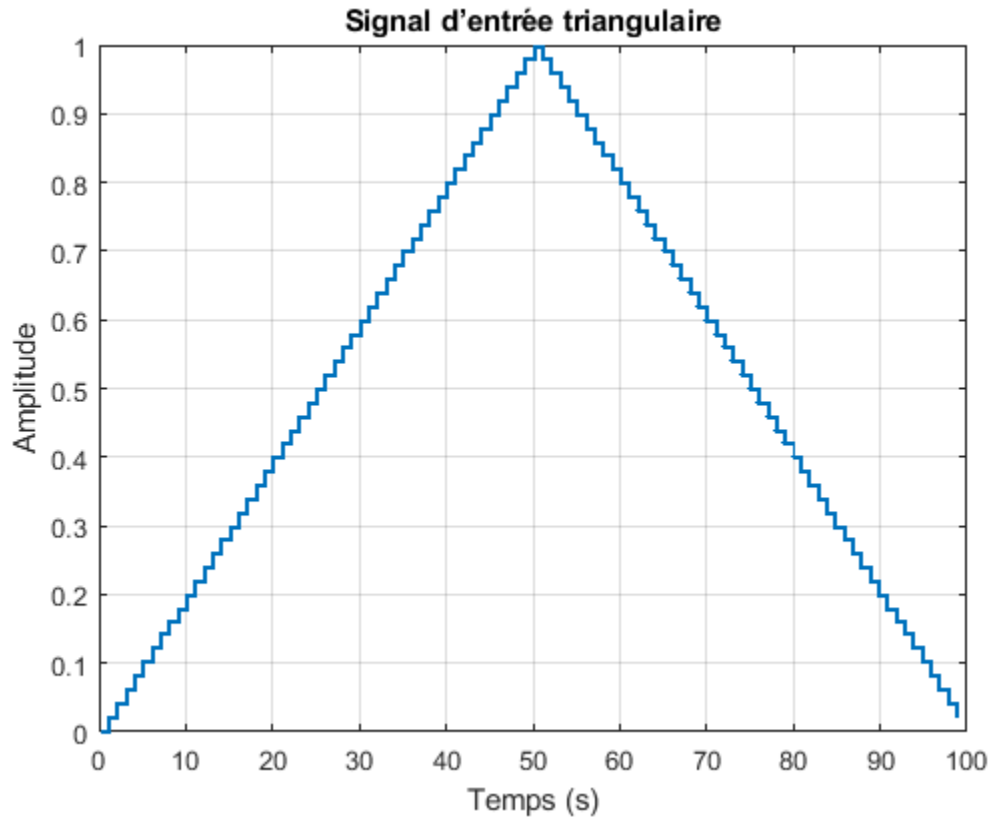
1. Construction du jeu de données

```
% Paramètres de discrétisation
L = 100; % Durée d'observation du signal d'entrée (secondes)
fe = 1; % Fréquence d'échantillonnage (Hz)
Te = 1/fe; % Période d'échantillonnage (secondes)
Nx = L/Te; % Nombre d'échantillons du signal x
nx = 0:Nx-1; % Indice temporel (entier)
tx = nx * Te; % Base de temps (secondes)
```

1a. Simulation du signal d'entrée triangulaire

```
x1 = (2/L) * tx(tx <= L/2); % Partie croissante
x2 = (-2/L) * tx(tx > L/2) + 2; % Partie décroissante
x = [x1 x2];
```

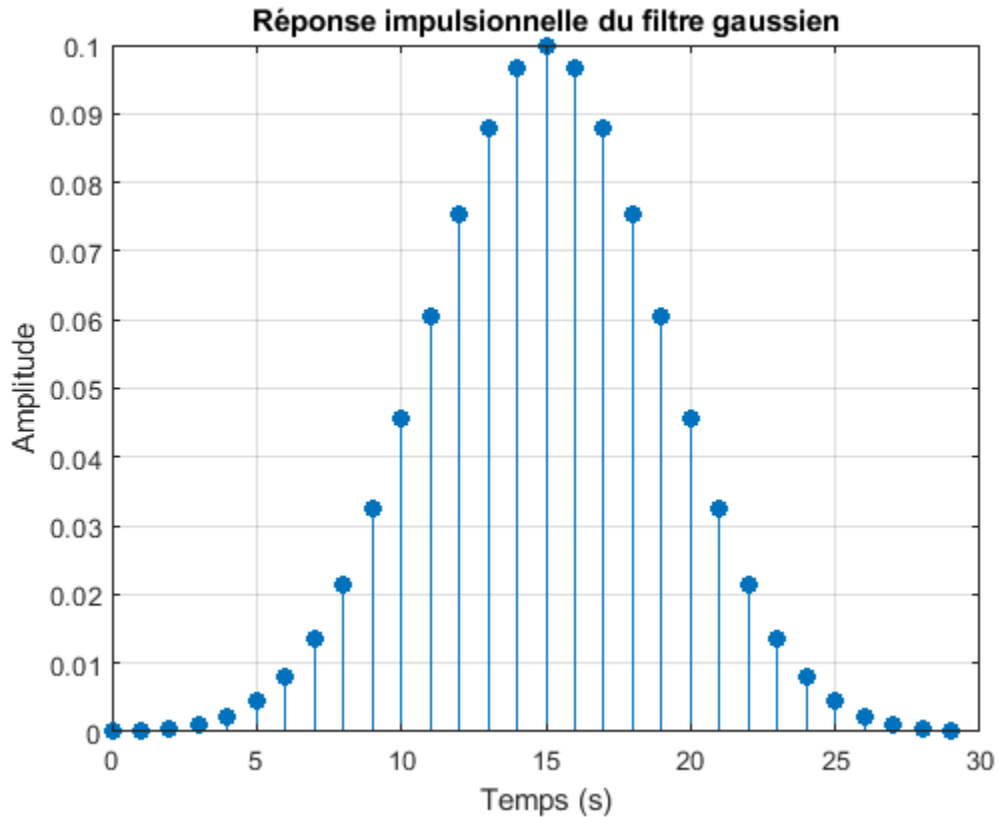
```
figure;
stairs(tx, x, 'LineWidth', 1.5);
title('Signal d'entrée triangulaire');
xlabel('Temps (s)'); ylabel('Amplitude'); grid on;
```



1b. Simulation du filtre gaussien

```
m = 15;  
sigma = 4;  
Nh = 30;  
th = 0:Nh-1;  
h = (1/(sigma*sqrt(2*pi))) * exp(-(th-m).^2 / (2*sigma^2));
```

```
figure;  
stem(th, h, 'filled');  
title('Réponse impulsionnelle du filtre gaussien');  
xlabel('Temps (s)'); ylabel('Amplitude'); grid on;
```



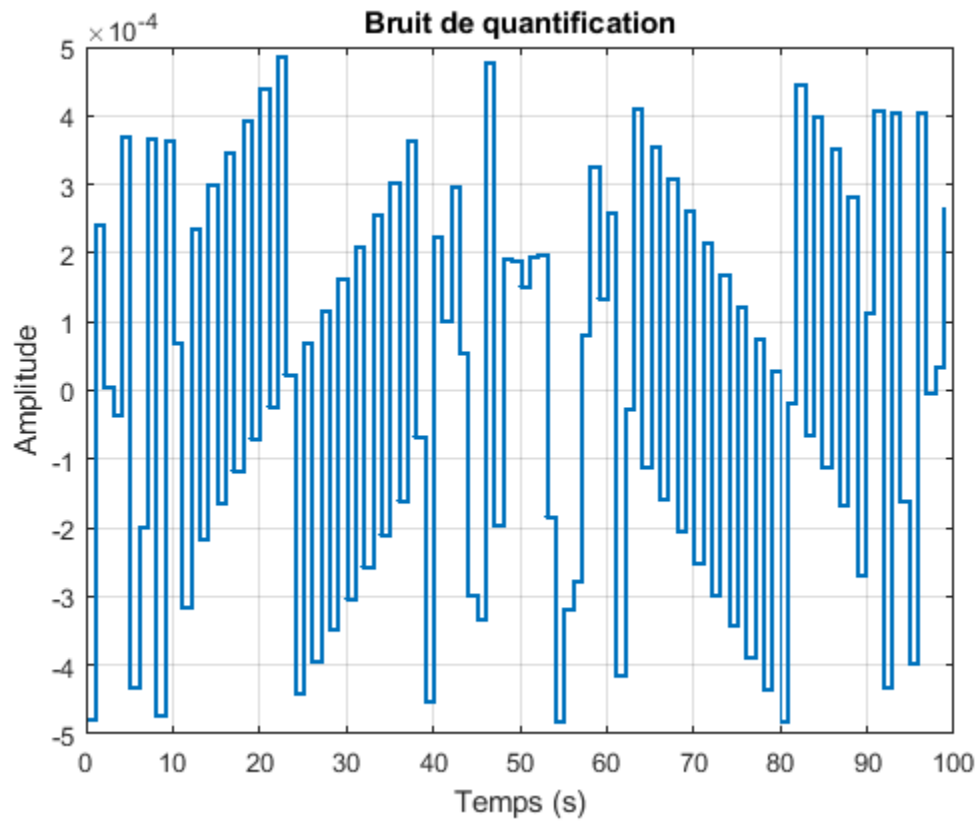
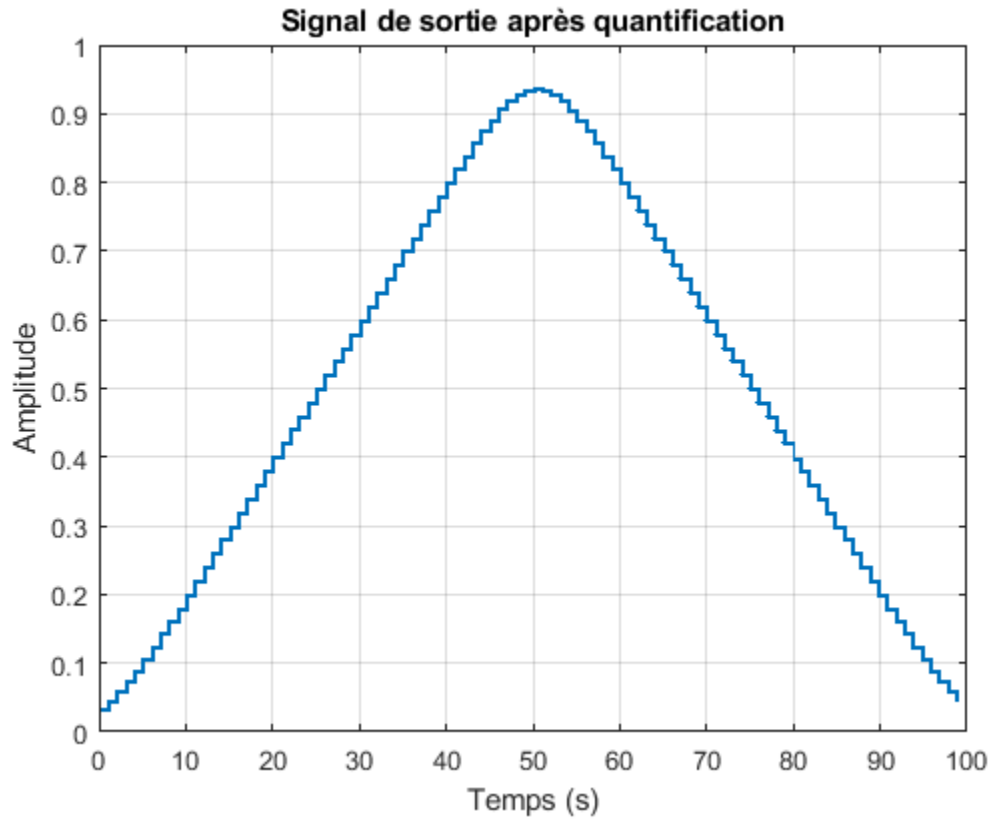
1c. Simulation du problème direct par convolution

```
ynb = conv(x, h, 'same');

% Ajout du bruit de quantification
y = round(ynb * 2^10) / 2^10;

% Représentation du signal de sortie bruité
figure;
stairs(tx, y, 'LineWidth', 1.5);
title('Signal de sortie après quantification');
xlabel('Temps (s)'); ylabel('Amplitude'); grid on;

% Détermination du bruit de quantification
b = y - ynb;
figure;
stairs(tx, b, 'LineWidth', 1.5);
title('Bruit de quantification');
xlabel('Temps (s)'); ylabel('Amplitude'); grid on;
```



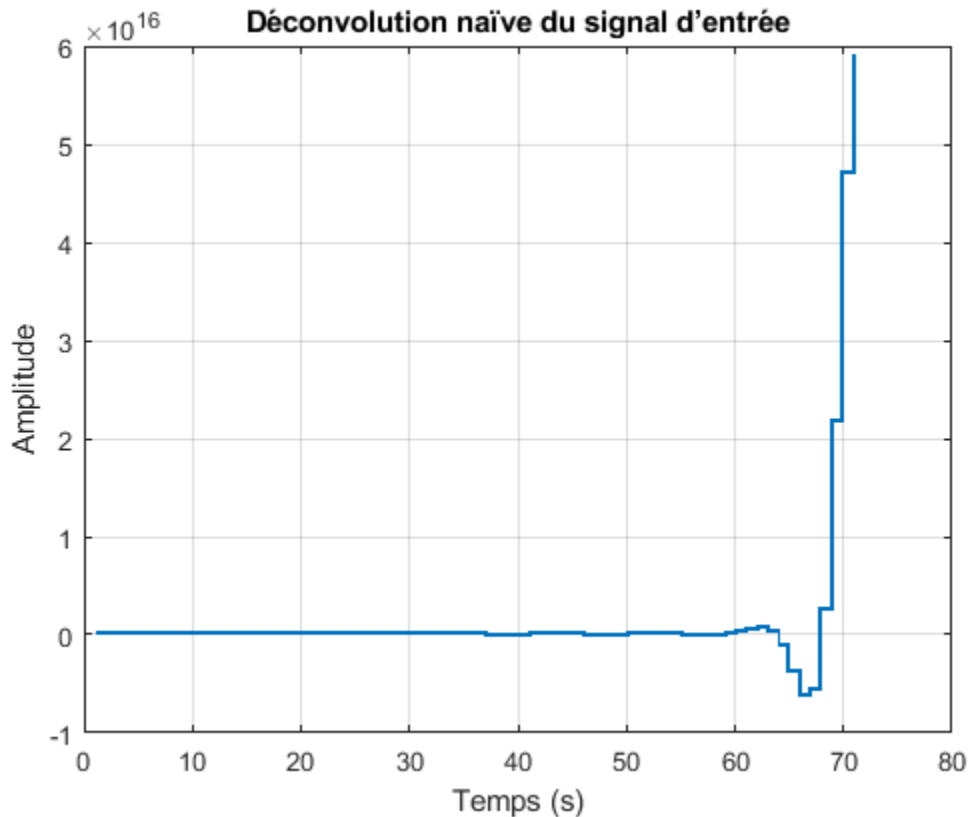
1d. Simulation matricielle du problème direct

```
Ny = Nx;  
H = toeplitz([h zeros(1, Ny-Nh)], [h(1) zeros(1, Ny-1)]);  
ynb_mat = H * x';
```

2a. Déconvolution naïve

```
[x_rec, ~] = deconv(ynb, h);
```

```
figure;  
stairs(x_rec, 'LineWidth', 1.5);  
title('Déconvolution naïve du signal d'entrée');  
xlabel('Temps (s)'); ylabel('Amplitude'); grid on;
```

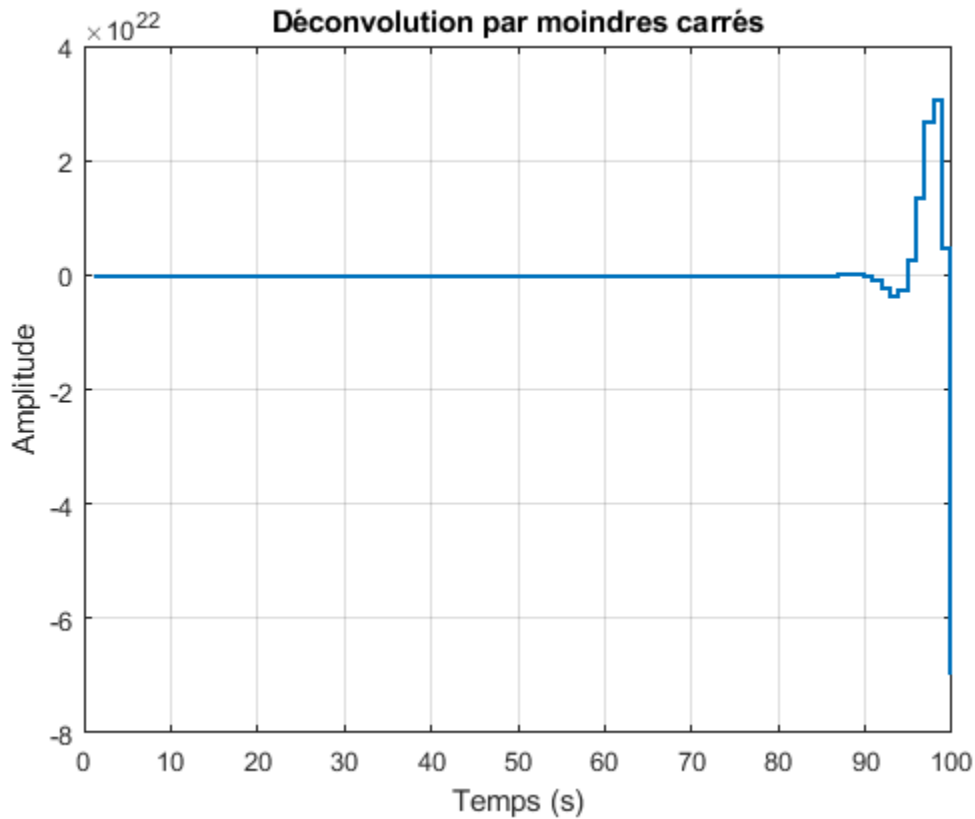


2b. Déconvolution par moindres carrés

```
x_ls = H \ y';
```

```
figure;  
stairs(x_ls, 'LineWidth', 1.5);  
title('Déconvolution par moindres carrés');  
xlabel('Temps (s)'); ylabel('Amplitude'); grid on;
```

```
% Conditionnement de H
s = svd(H);
kappa = max(s) / min(s(s > 0));
```



2c. Déconvolution par régularisation linéaire quadratique

```
D1 = toeplitz([1 -1 zeros(1, Nx-2)], [1 zeros(1, Nx-1)]);
lambda_vals = logspace(-7, 2, 50);
errors = zeros(size(lambda_vals));

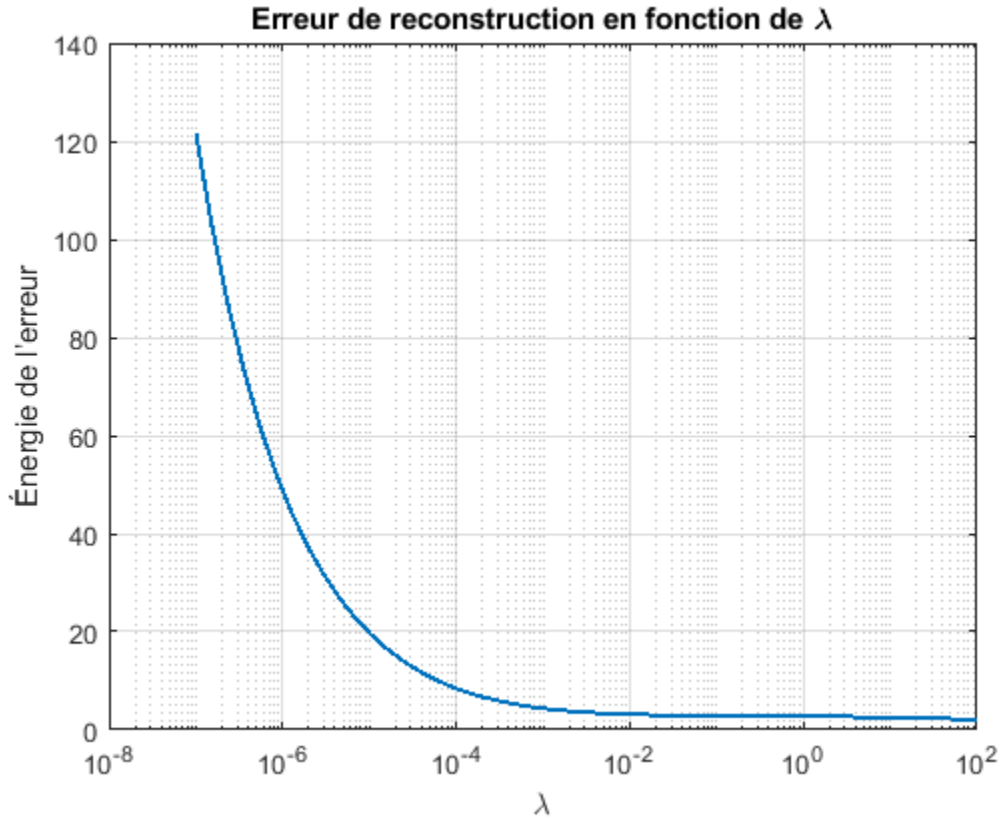
for i = 1:length(lambda_vals)
    lambda = lambda_vals(i);
    x_reg = (H'*H + lambda * D1'*D1) \ (H'*y);
    errors(i) = norm(x - x_reg', 2);
end

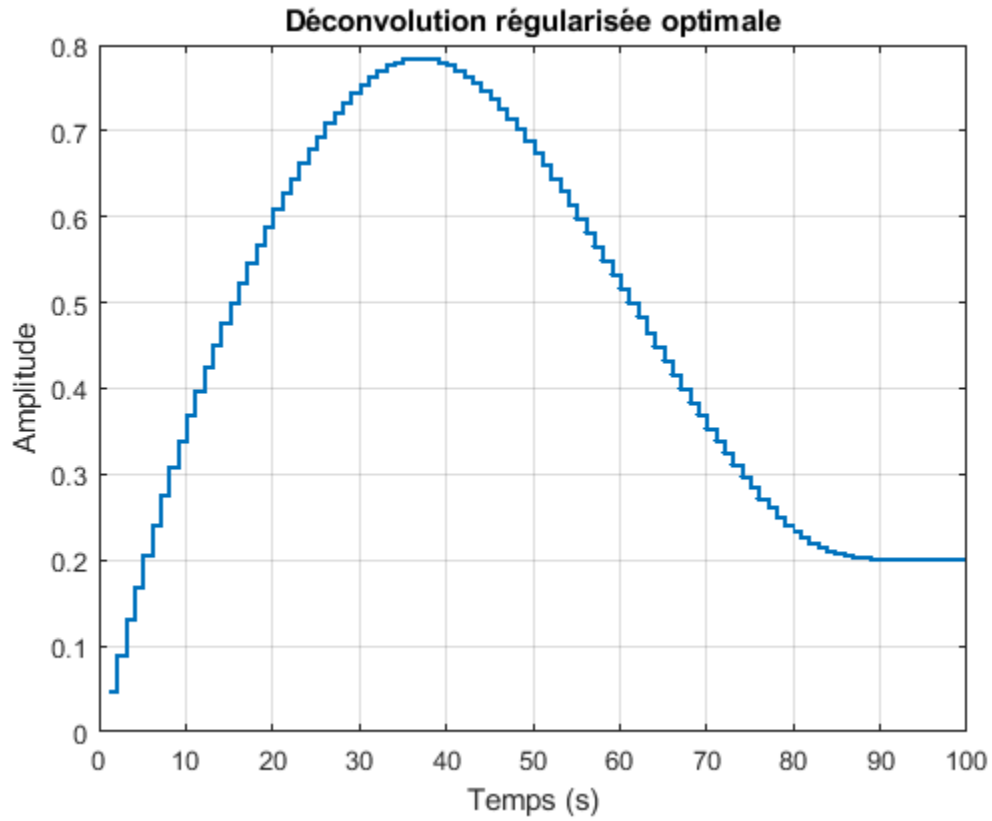
figure;
semilogx(lambda_vals, errors, 'LineWidth', 1.5);
title('Erreur de reconstruction en fonction de \lambda');
xlabel('\lambda'); ylabel('Énergie de l'erreur'); grid on;

% Meilleure valeur de lambda
[~, idx] = min(errors);
lambda_opt = lambda_vals(idx);
```

```
x_opt = (H'*H + lambda_opt * D1'*D1) \ (H'*y');

figure;
stairs(x_opt, 'LineWidth', 1.5);
title('Déconvolution régularisée optimale');
xlabel('Temps (s)'); ylabel('Amplitude'); grid on;
```





Published with MATLAB® R2024a